

War Game and Security Assessment Report

Team 09

Team Member Details

(1) Deependra Singh (CS25MTECH14017)
(2) Rohit B (CS25MTECH14012)
(3) Yash Vardhan (CS25MTECH14016)
(4) Hemang Ranga (CS25 MTECH 14005)
(5) Manan Kannasey (CS25MTECH11028)

Transaction War Phase 2 Report

(1) Introduction –

The Introduction section contains the functionality and core security mechanism of our web application -

(A) Functionality of Our Web Application –

MoneyFlow is a web-based financial transaction platform built using PHP 8.2, MySQL, and vanilla HTML/CSS/JavaScript, deployed inside a Docker container environment. The application provides the following core features:

(a) User Registration & Initial Credit: Users can register by providing a unique username, email address, and a strong password. A server-side math CAPTCHA must be solved during registration to prevent automated signups. Upon successful registration, each user is automatically credited with an initial balance of Rs. 100.

(b) Login & Logout: Registered users can log in using their credentials after solving a math CAPTCHA challenge. The logout functionality is restricted to POST requests with CSRF token validation to prevent forced-logout attacks.

(c) Session Management: The application maintains user sessions through a fully custom token-based system rather than relying on PHP's default session mechanism. This gives us tighter control over session lifecycle, expiration, and binding to client attributes.

(d) Profile Management: Users can update their biography (plain text, up to 500 characters) and upload a profile picture. Usernames, email addresses, and passwords cannot be changed through the profile settings page. Users can also view other members' profiles, though sensitive fields like email are only visible on one's own profile.

(e) User Search: Members can search for other users by username or user ID. Search results are paginated and the search functionality is rate-limited to prevent abuse.

(f) Money Transfer with Step-Up Authentication: Users can transfer money to other members by specifying the recipient's user ID and the amount. Before performing any transfer, the user must re-enter their password as a step-up authentication measure, which remains valid for a 15-minute window. An optional comment can be attached to each transfer, visible to the receiver. The system prevents negative balance transactions and self-transfers.

(g) Transaction History: A paginated transaction history page displays all past transfers — both sent and received — along with timestamps, amounts, counterparty details, and any attached comments.

(h) Activity Logging: Every page access across the application is logged with the associated username, page path, client IP address, User-Agent string, and timestamp for audit and forensic purposes.

(i) Dockerized Deployment: The entire application runs via docker-compose with two containers — one for the Apache web server and one for the MySQL database. Apache serves the application over HTTPS on port 8443, with HTTP requests on port 8080 automatically redirected to the secure endpoint. A self-signed TLS certificate is generated at build time for encrypted communication.

(B) Core Security Mechanism –

The following security mechanism applied by our team on the web application -

(a) Password Hashing with Argon2id: All passwords are hashed using the Argon2id algorithm, configured with 64 MB memory cost, 4 time passes, and 2 parallel threads. This makes brute-force and GPU-based cracking attacks computationally expensive. Verification uses PHP's timing-safe password_verify() to prevent timing side-channel leaks. The system also auto-upgrades older bcrypt hashes to Argon2id upon successful login.

(b) Custom Session Token System: Session tokens are generated from 32 cryptographically secure random bytes, producing 64-character hex strings. Only the SHA-256 hash of each token is stored in the database, protecting raw tokens even if the database is compromised. Each session is bound to the client's IP address and a SHA-256 hash of their User-Agent string to prevent hijacking. A 30-minute sliding inactivity timeout and a hard 2-hour absolute lifetime are enforced. On every new login, all previous sessions for that user are invalidated and PHP's internal session ID is regenerated to prevent fixation attacks. Session cookies carry HttpOnly, Secure, and SameSite=Strict flags.

(c) CSRF Protection: Every state-changing form includes a CSRF token generated from cryptographically secure random bytes and stored server-side in the session record. Validation uses timing-safe hash_equals() comparison. Even the logout action requires a valid CSRF token submitted via POST to prevent forced-logout through embedded image tags or similar vectors.

(d) Server-Side Math CAPTCHA: Both the login and registration pages require users to solve a server-generated math challenge (random addition of two numbers). The correct answer is stored exclusively in the PHP session and is never sent to the client. The answer is consumed and regenerated immediately after each submission attempt, preventing replay attacks.

(e) SQL Injection Prevention: All database queries use PDO prepared statements with named parameter binding. Emulated prepares are disabled (ATTR_EMULATE_PREPARES = false) so the database driver handles

parameterization natively. Multi-statement execution is blocked (MYSQL_ATTR_MULTI_STATEMENTS = false) to prevent stacked query injection. Wildcard characters in LIKE clauses are manually escaped before binding.

(f) XSS Prevention & Output Encoding: All user-supplied data rendered on any page passes through htmlspecialchars() with ENT_QUOTES | ENT_HTML5 and UTF-8 encoding. Biographies are stored and displayed as plain text only. A strict Content Security Policy header restricts script sources to self — no inline scripts, no data: URIs, and no unsafe-inline or unsafe-eval are allowed. All JavaScript is loaded from external files only.

(g) Secure Money Transfer via Database Intent Binding: The transfer mechanism uses a database intent binding pattern to defeat proxy-level tampering tools like Burp Suite. When a user initiates a transfer, the server validates the inputs and writes a pending record to the database, then encrypts the transaction ID along with sender, recipient, amount, and a nonce using AES-256-GCM authenticated encryption with a random 12-byte IV. The client submits this encrypted token back, and the server decrypts it, verifies it against the database record, and executes the transfer using only the database-stored values — never the client-submitted data. A session-stored nonce is checked against the token to prevent replay, and the pending record is atomically marked as used with a row count check to guarantee only one request can execute a given transfer, preventing double-spend.

(h) ACID Transactions with Deadlock Prevention: All money transfers execute within a database transaction using BEGIN, COMMIT, and ROLLBACK. Both the sender's and recipient's rows are locked using SELECT ... FOR UPDATE, with locks always acquired in ascending ID order regardless of transfer direction. This consistent lock ordering prevents bidirectional deadlocks when two users transfer to each other simultaneously.

(i) Database-Level Balance Constraint: A CHECK (account_balance >= 0) constraint is enforced at the MySQL level on the members table. This acts as a last line of defense against negative balances, ensuring that even if application-level validation is somehow bypassed, the database itself will reject the transaction.

(j) Step-Up Re-Authentication for Transfers: Before executing any financial transaction, the user must re-enter their password. This re-authentication is valid for a 15-minute window and is rate-limited to 5 failed attempts per IP within 15 minutes. This protects against session-riding attacks where an attacker gains access to an already-authenticated session.

(k) Multi-Layer Rate Limiting: Login attempts are capped at 5 failures per IP-username pair within 2 hours, and 10 failures per username globally within 6 hours to catch attackers rotating IPs. Registration is limited to 5 accounts per IP per 24 hours. Search is capped at 10 queries per IP per minute. Transfers are limited to 10 per user per day with a Rs. 5,000 daily outflow cap. Avatar uploads are restricted to 10 per user per hour. All rate limit checks use atomic INSERT-SELECT queries to avoid

time-of-check-to-time-of-use race conditions, and stale records are auto-cleaned by scheduled MySQL events.

(l) **File Upload Security:** Profile image uploads are validated through multiple layers — MIME type checking via finfo (reading actual file bytes, not the filename), image structure verification using getimagesize() to catch polyglot files, a 2 MB size limit, and a 4000x4000 pixel dimension cap. Every uploaded image is then re-encoded through PHP's GD library, which strips any embedded payloads, EXIF metadata, and malicious content. Saved filenames are randomly generated using random_bytes() and never derived from user input. The file extension is mapped from the validated MIME type, not the original filename. PHP script execution is explicitly disabled in the uploads directory at the Apache configuration level.

(m) **Host Header Validation:** Apache is configured to accept only IPv4 addresses in the Host header, blocking Host header injection attacks that could lead to cache poisoning, password reset poisoning, or web-cache deception.

(n) **Sensitive Directory Access Blocking:** Apache rewrite rules block direct access to dot files and directories (.git, .env, etc.) and backend directories like /settings, /utilities, and /layouts, preventing source code disclosure or configuration leakage.

(o) **Server Hardening & Security Headers:** Apache hides its version (ServerTokens Prod, ServerSignature Off) and PHP's expose_php is off. The application sets X-Content-Type-Options: nosniff, X-Frame-Options: DENY against clickjacking, Strict-Transport-Security with one-year max-age and preload, Referrer-Policy: strict-origin-when-cross-origin, and a Permissions-Policy disabling camera, microphone, geolocation, and payment APIs. TLS 1.3 is enforced exclusively, rejecting all older protocol versions. SSL compression and session tickets are disabled to prevent CRIME/BREACH attacks and improve forward secrecy.

(p) **Dangerous Function Restrictions & Sandboxing:** A broad set of dangerous PHP functions — including system, exec, shell_exec, passthru, proc_open, popen, curl_exec, phpinfo, mail, putenv, and others — are disabled in the PHP configuration. The open_basedir directive restricts PHP's filesystem access to /var/www/html and /tmp only. The container runs on unprivileged ports (8080/8443) and operates under the www-data user rather than root.

(q) **Container Resource Limits & Network Isolation:** Each Docker container is limited to 1 CPU and 512 MB of RAM, preventing resource exhaustion attacks — particularly important since each Argon2id hash consumes 64 MB of memory. The MySQL database port is not exposed to the host network and is accessible only within the Docker internal network, reducing the attack surface.

(r) **No Hardcoded Secrets:** All sensitive values — HMAC signing key, AES-256 encryption key, and database passwords — must be supplied through environment variables. The HMAC key must be at least 32 characters and the AES key must be exactly 64 hex characters (32 bytes). The application refuses to start if any required secret is missing or does not meet length requirements.

(s) **Pagination Capping:** The transaction history page caps the maximum page number at 10,000 (covering up to 200,000 records at 20 per page). This prevents attackers from requesting absurdly high page numbers that would cause MySQL to perform massive OFFSET scans, which could degrade database performance.

(t) **Activity Logging & Audit Trail:** Every page access is recorded in an event_chronicle database table, capturing the member ID, username, page path, client IP, User-Agent, and timestamp. Both authenticated and unauthenticated requests are logged. Login successes and failures are explicitly tracked. Records older than 90 days are automatically purged by a scheduled database event.

(u) **Response Integrity Verification:** The application computes an HMAC-SHA256 hash of the entire HTTP response body and attaches it as an X-Response-Integrity header, enabling detection of any response tampering that might occur in transit between the server and the client.

(v) **Database Privilege Restriction:** The MySQL application user is granted only SELECT, INSERT, UPDATE, and DELETE privileges after initial schema setup. All higher-level privileges such as CREATE, DROP, and ALTER are explicitly revoked, limiting the potential damage even in the event of a successful SQL injection attack.

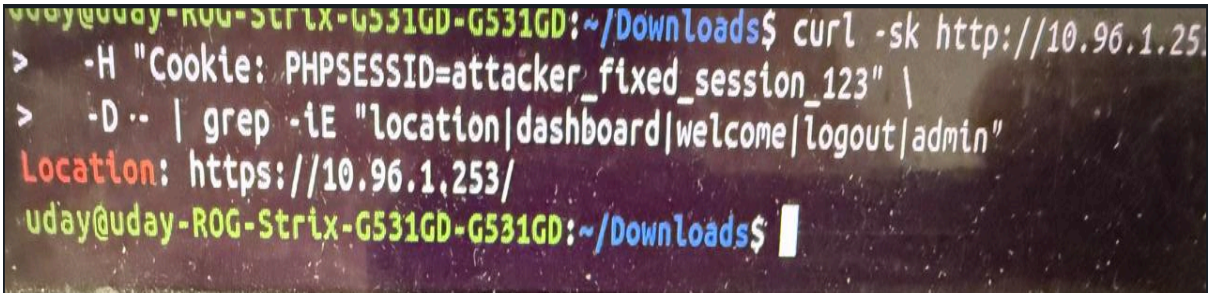
(2) War Game Attack Analysis –

No team was able to successfully exploit any valid security vulnerability in our application during the entire War Game phase. However, for the sake of completeness and transparency, we have documented all the reported attempts below along with our analysis of each.

(A) Attacks Conducted by Other Teams on our application:

(1) **Attack by –** Team 06
Type of Attack – Session Fixation - Pre-Login Session ID Accepted After Authentication.
Severity and potential impact – The attack had zero impact since our application uses a custom token-based session system, not PHP's default PHPSESSID. The pre-planted session ID was simply ignored by the server.
Successful or mitigated – This is not even a bug. The server never accepted or authenticated the attacker's crafted session ID. Our application generates a fresh cryptographically random token via random_bytes(32) on every login and invalidates all previous sessions from the database. The attacker's screenshot only showed an HTTP-to-HTTPS 301 redirect with no Set-Cookie header — no authentication occurred.

Validity – Invalid. The attacker misunderstood our session architecture and assumed the application relies on PHP's native PHPSESSID for authentication, which it does not.



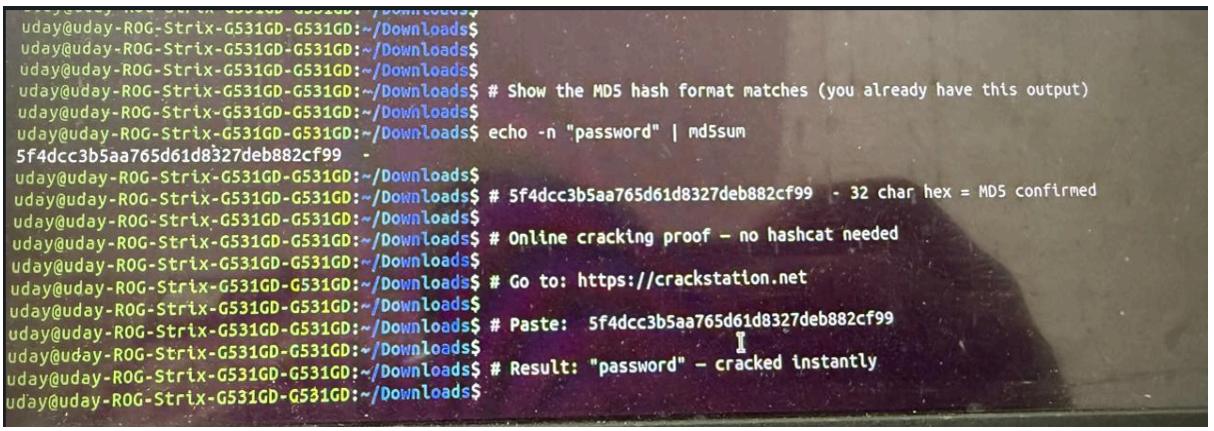
(2) Attack by – Team 06

Type of Attack – Weak Password Hashing - MD5 used for password Storage (False claim)

Severity and Potential Impact – The claim is entirely fabricated. Our application uses Argon2id for password hashing, not MD5. The attacker's screenshot simply shows them running echo -n "password" | md5sum on their own terminal and looking up that MD5 hash on CrackStation — this has nothing to do with our application or database.

Successful or Mitigated – This is not even a bug. Not applicable. The attack never happened on our system. The attacker provided no evidence of extracting any hash from our database. They generated an MD5 hash of the word "password" on their own machine and presented it as proof.

Validity – Invalid. Completely false claim. Our application uses Argon2id with 64 MB memory cost, 4 time passes, and 2 threads for all password storage. No MD5 is used anywhere in our system.



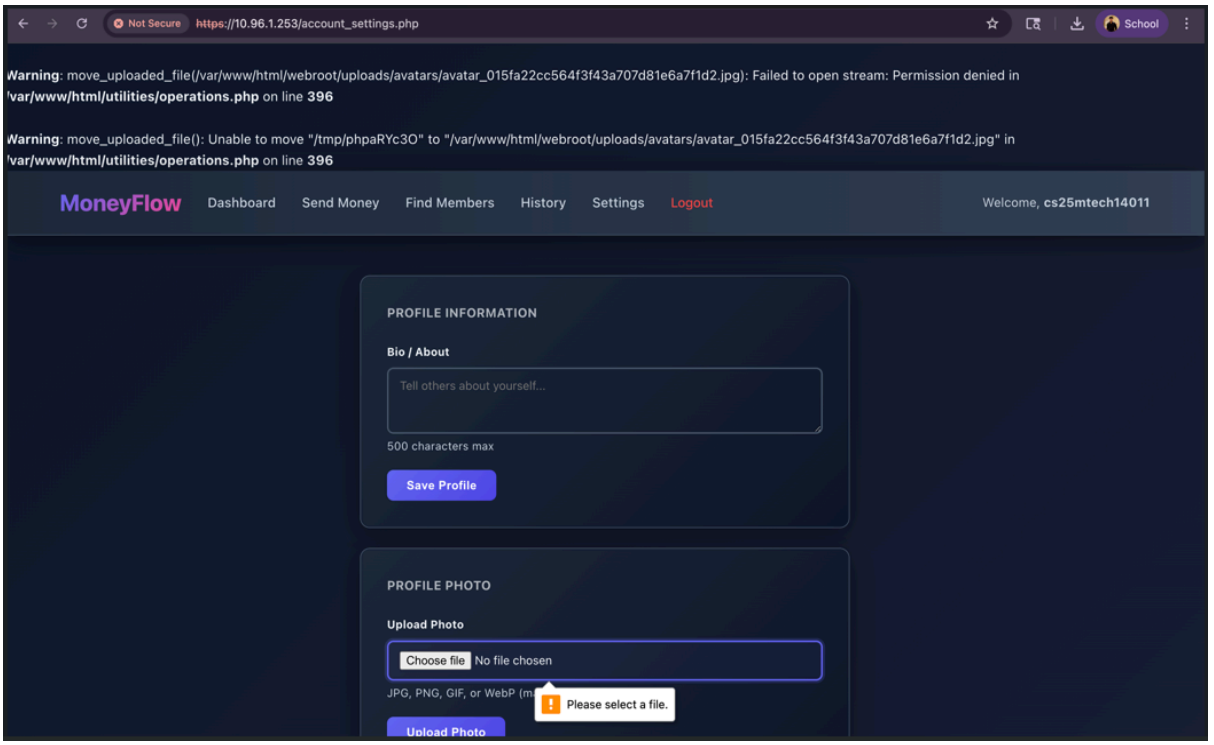
(3) **Attack by –** Team 07

Type of Attack – Improper file handling due to insufficient File Permissions (Information disclosure through verbose error messages exposed during file upload.)

Severity and Potential Impact – The error was a deployment-time file permission misconfiguration on the uploads directory, not a security vulnerability in the application code. The exposed internal paths (/var/www/html/webroot/uploads/avatars/, /utilities/operations.php) are standard default paths and do not reveal any sensitive information that an attacker could leverage.

Successful or Mitigated – This is not even a bug. This was a temporary server configuration issue, not an exploitable vulnerability. The file upload logic itself — including MIME validation, GD re-encoding, and random filename generation — was fully intact. The permission issue was resolved by correcting directory ownership during deployment.

Validity – Invalid as a security attack. This is a deployment configuration issue, not a vulnerability in the application. The error messages were visible because display_errors was on during initial deployment and was turned off afterwards. The underlying upload security mechanisms were never compromised.



(4) Attack by – Team 03

Type of Attack – Profile picture upload failed despite valid file size.

Severity and Potential Impact – The upload was correctly rejected. Our application does not validate files based on size alone — it also checks MIME type via finfo, verifies image structure using getimagesize(), and re-encodes through GD library. The file the attacker uploaded appears to be a blank or corrupted file (their own screenshot shows a completely white/empty image) that likely failed the image structure validation, not the size check. The error message displayed was the size error but the rejection was triggered by the file not being a valid image.

Successful or Mitigated –This is not even a bug. Not applicable. The application behaved as intended by rejecting a file that did not pass image validation checks.

Validity – Invalid. The attacker assumed file size is the only validation criterion. Our upload pipeline validates MIME type, image structure, dimensions, and then re-encodes the image through GD. A file that is not a genuine image will be rejected regardless of its size.



This attacker tried to upload this white image with less than 2 MB size but more pixel

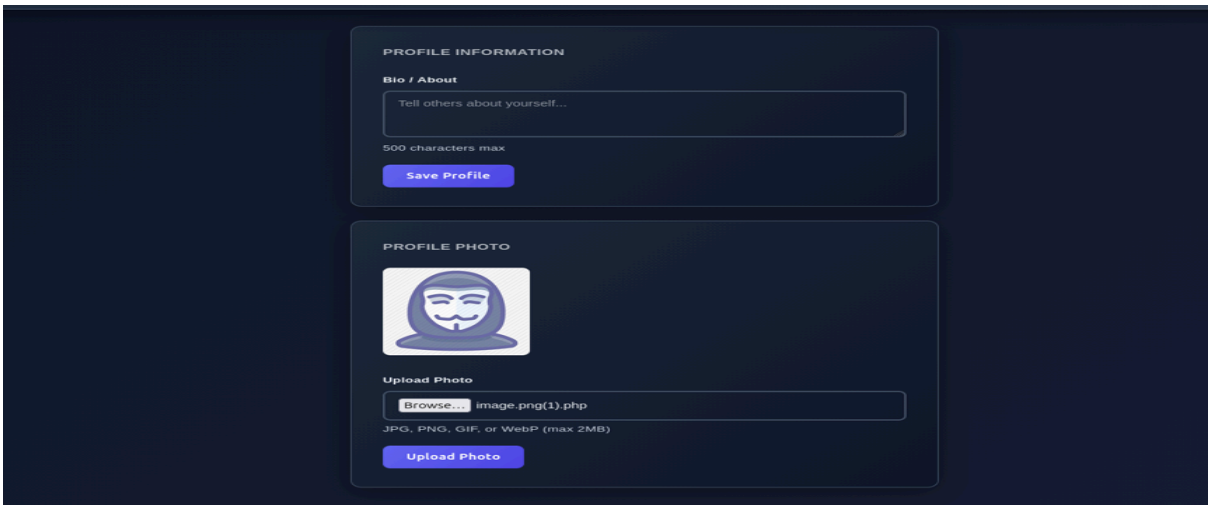
(5) Attack by – Team 15

Type of Attack – Defective Image with Malicious Script file can be uploaded(Malicious file upload attempting to execute PHP code on the server by disguising a script with a double extension (image.png.php).

Severity and Potential Impact – The uploaded file was not a PHP script — it was a real image file with a tampered extension. Our application never trusts the filename or extension provided by the user. The server reads actual file bytes using finfo to determine the real MIME type, validates image structure with getimagesize(), re-encodes through GD library, and saves with a randomly generated filename mapped from the validated MIME type. Even if a genuine PHP file were uploaded, finfo would detect it as text/plain, not an image, and reject it immediately. Additionally, PHP execution is disabled in the uploads directory at the Apache level.

Successful or Mitigated – This is not even a bug. Mitigated. The file that got uploaded was simply a real image with a manipulated extension — our system stripped the original filename entirely and saved it with a random name and correct extension. No malicious code was ever stored or executed.

Validity – Invalid. The attacker confused a tampered filename with actual malicious content. The screenshot itself shows the avatar displaying correctly, proving it was a genuine image all along. Our upload pipeline validates file content, not filenames.



(6) Attack by – Team 16

Type of Attack – SQL Injection Authentication Bypass on Login Form(SQL Injection on the login form using classic ' OR '1'='1' -- payload to bypass authentication.)

Severity and Potential Impact – SQL injection is structurally impossible in our application. All database queries use PDO prepared statements with named parameter binding, emulated prepares are disabled (ATTR_EMULATE_PREPARES = false), and multi-statement execution is blocked. User input never touches the SQL query string.

Successful or Mitigated – This is not even a bug. Mitigated by design. The attacker provided no proof of successful login — no screenshot of a dashboard, no session cookie, no authenticated page. Their "proof" is just a curl command they wrote, not evidence of actual bypass. Furthermore, the report references /login.php which does not exist in our application — our login page is signin.php. Entering the payload ' OR '1'='1' -- in the username field simply returns "Invalid Credentials."

Validity – Invalid. The attacker fabricated the claim. The referenced endpoint does not exist, no proof of successful authentication was provided, and the underlying architecture makes SQL injection impossible regardless of the payload used.

```
Proof of Concept: SQL Injection Authentication Bypass
Target: Team 9 (10.96.1.253)
URL: https://10.96.1.253:443/login.php
Date: 2026-03-19

== REPRODUCTION ==

Step 1: Navigate to https://10.96.1.253:443/login.php
Step 2: In username field, enter: ' OR '1'='1' --
Step 3: Enter any password
Step 4: Submit
Step 5: Observe: Authenticated without valid credentials

== CURL PROOF ==

TOKEN=$(curl -k -c cookies.txt -s https://10.96.1.253:443/login.php | grep -oP 'value="\K[^"]+(?="[^>]*name="csrf)')
curl -k -b cookies.txt -c cookies.txt -L -v -X POST https://10.96.1.253:443/login.php \
  -d "username=' OR '1'='1' --&password=anything&csrf_token=$TOKEN"

== IMPACT ==
High - Full auth bypass. Access any account without credentials.
```

(7) Attack by – Team 16

Type of Attack – Default Admin credentials grant full access(Credential guessing using default admin credentials (admin:admin).)

Severity and Potential Impact – No such account exists in our application. Our registration system enforces strong password requirements — minimum 8 characters with mandatory uppercase, lowercase, digits, and special characters. A password like "admin" cannot even be created in our system as it fails every validation rule.

Successful or Mitigated – This is not even a bug. Not applicable. The attack never succeeded. Attempting to log in with admin:admin simply returns "Invalid Credentials." The attacker again references /login.php which does not exist — our login page is signin.php. No proof of successful authentication was provided.

Validity – Invalid. Completely fabricated claim with no supporting evidence. No default admin account exists, and the password policy makes it impossible to create an account with such a weak password.

```
Proof of Concept: Default Admin Credentials
Target: Team 9 (10.96.1.253)
URL: https://10.96.1.253:443/login.php
Date: 2026-03-19

== REPRODUCTION ==

Step 1: Navigate to https://10.96.1.253:443/login.php
Step 2: Username: admin  |  Password: admin
Step 3: Click Login
Step 4: Observe: Full admin access granted

== IMPACT ==
High - Trivial to exploit. Full admin access.
```

(8) Attack by – Team 13

Type of Attack – Able to upload .png.php file in profile

Severity and Potential Impact – Our application does not rely on file extensions for validation at all. The server reads actual file bytes using finfo to determine the real MIME type, validates image structure with getimagesize(), and re-encodes through GD. The saved filename is randomly generated and the extension is derived from the validated MIME type — the original filename is completely discarded.

Successful or Mitigated – This is not even a bug. Not applicable. The attacker provided no screenshot or proof that the upload was accepted. A .png.php file containing PHP code would be detected as text/plain by finfo and rejected immediately. Even if a real image with a .png.php extension were uploaded, it would be re-encoded, saved with a random name and correct extension, and PHP execution is disabled in the uploads directory via Apache configuration.

Validity – Invalid. No proof of concept was provided. The claim was made without any supporting evidence — no screenshot of a successful upload, no demonstration of code execution, nothing.

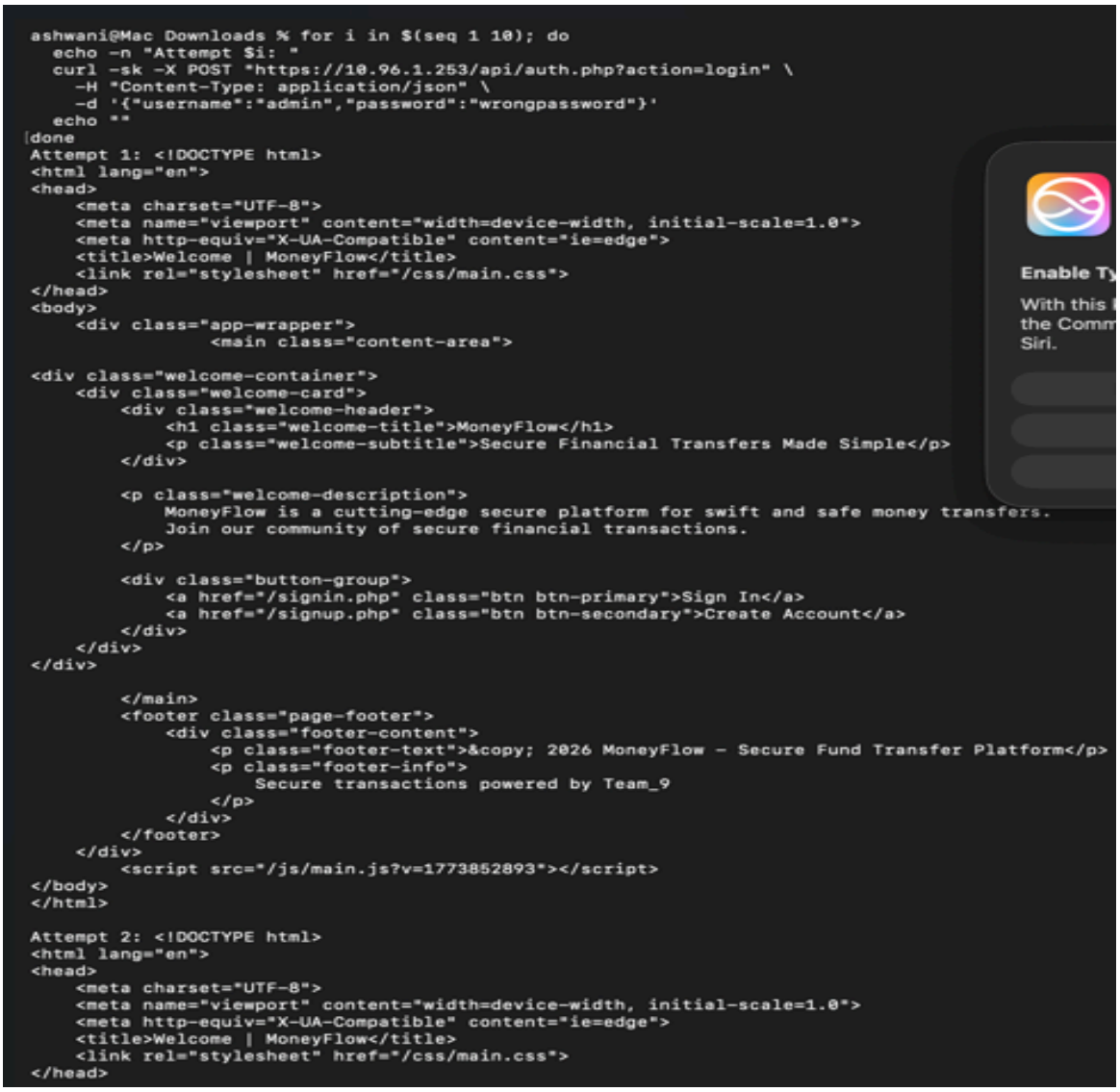
(9) Attack by – Team 18

Type of Attack – No rate limiting on login endpoint - Brute force attack possible(Brute force / credential stuffing by sending repeated login attempts in a loop.)

Severity and Potential Impact – Attacker tested against /api/auth.php?action=login which does not exist in our application. Every request simply returned the welcome page HTML — no authentication logic was ever invoked. Our actual login endpoint (signin.php) has multi-layer rate limiting already in place.

Successful or Mitigated – This is not even a bug.Not applicable. The attack was performed against a non-existent endpoint. The response body in the attacker's own screenshot shows the MoneyFlow welcome page HTML with "Sign In" and "Create Account" buttons — confirming no login processing occurred. Our real login at signin.php enforces 5 failed attempts per IP-username pair within 2 hours, 10 failed attempts per username globally within 6 hours, and requires a server-side math CAPTCHA on every attempt.

Validity – Invalid. Wrong endpoint tested. The attacker's own screenshot serves as proof that the attack never reached any authentication logic.



```
ashwani@Mac Downloads % for i in $(seq 1 10); do
  echo -n "Attempt $i: "
  curl -sk -X POST "https://10.96.1.253/api/auth.php?action=login" \
    -H "Content-Type: application/json" \
    -d '{"username":"admin","password":"wrongpassword"}'
  echo ""
done
Attempt 1: <!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Welcome | MoneyFlow</title>
    <link rel="stylesheet" href="/css/main.css">
</head>
<body>
    <div class="app-wrapper">
                <main class="content-area">

<div class="welcome-container">
    <div class="welcome-card">
        <div class="welcome-header">
            <h1 class="welcome-title">MoneyFlow</h1>
            <p class="welcome-subtitle">Secure Financial Transfers Made Simple</p>
        </div>

        <p class="welcome-description">
            MoneyFlow is a cutting-edge secure platform for swift and safe money transfers.
            Join our community of secure financial transactions.
        </p>

        <div class="button-group">
            <a href="/signin.php" class="btn btn-primary">Sign In</a>
            <a href="/signup.php" class="btn btn-secondary">Create Account</a>
        </div>
    </div>
</div>

        </main>
        <footer class="page-footer">
            <div class="footer-content">
                <p class="footer-text">&copy; 2026 MoneyFlow - Secure Fund Transfer Platform</p>
                <p class="footer-info">
                    Secure transactions powered by Team_9
                </p>
            </div>
        </footer>
    </div>
        <script src="/js/main.js?v=1773852893"></script>
</body>
</html>

Attempt 2: <!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Welcome | MoneyFlow</title>
    <link rel="stylesheet" href="/css/main.css">
</head>
```

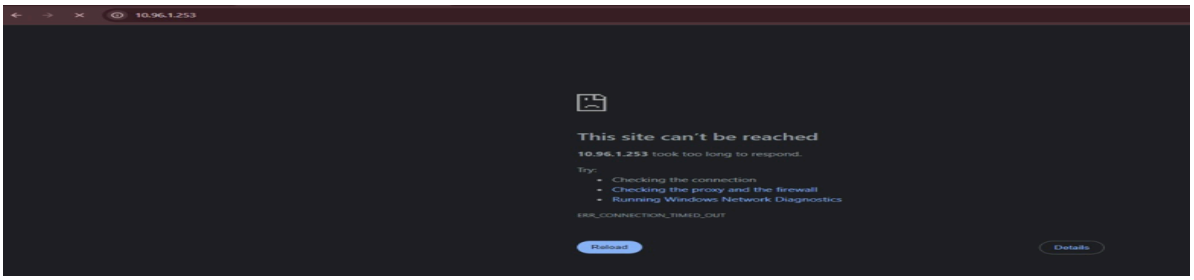
(10) Attack by – Team 12

Type of Attack – site down

Severity and Potential Impact – This was not even a bug, The attacker was using http://10.96.1.253 instead ofhttps://10.96.1.253.

Successful or Mitigated – Not a bug

Validity – Invalid.



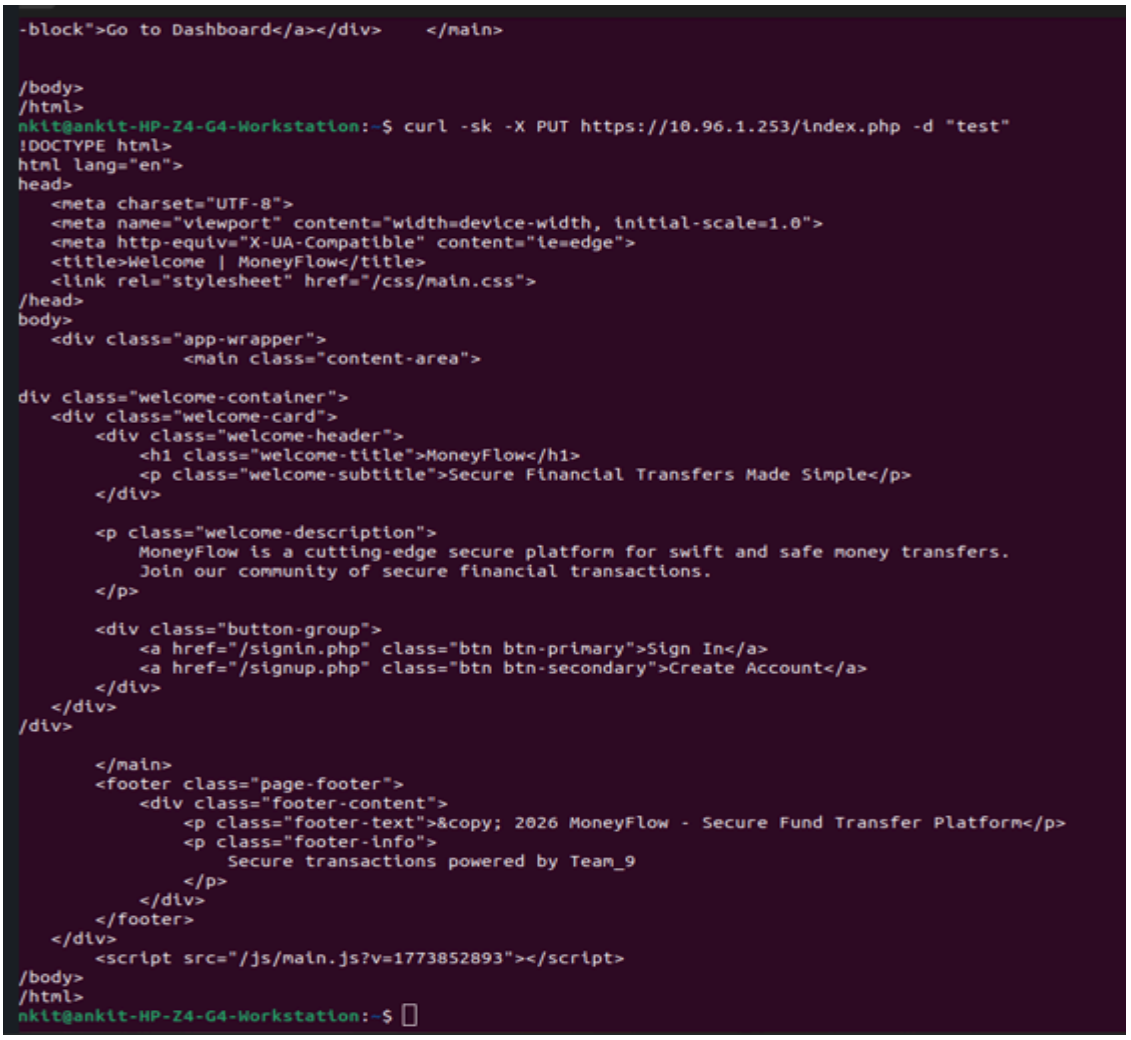
(11) Attack by – Team 05

Type of Attack – Support for Insecure HTTP method(put) and exposure of deployment meta data(HTTP method tampering (PUT request) and information disclosure via cache-busting query parameter.)

Severity and Potential Impact – The PUT request did not write or modify anything on the server. Apache's rewrite rule simply routed it to index.php, which served the same welcome page HTML and ignored the PUT body entirely. The ?v=1773852893 on the JavaScript file is a standard filemtime()-based cache-busting parameter — a common industry practice used by virtually every web application to ensure browsers load updated assets. It reveals no actionable information.

Successful or mitigated – This is not even a bug. Not applicable. No file was created, modified, or overwritten. The attacker's own screenshot proves this — the response is just the normal welcome page HTML. Receiving a 200 instead of 405 does not constitute a vulnerability when the request has zero side effects. No data was altered, no shell was uploaded, no state was changed.

Validity –Invalid. The attacker confused "server responded" with "server is exploitable." Returning 200 for an unsupported method with no side effects is not a security vulnerability. The cache-busting parameter claim is baseless — this is standard web development practice, not metadata leakage.



```
-block">Go to Dashboard</a></div>    </main>


/body>
/html>
nkit@ankit-HP-Z4-G4-Workstation:~$ curl -sk -X PUT https://10.96.1.253/index.php -d "test"
!DOCTYPE html>
html lang="en">
head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <meta http-equiv="X-UA-Compatible" content="ie=edge">
   <title>Welcome | MoneyFlow</title>
   <link rel="stylesheet" href="/css/main.css">
/head>
body>
   <div class="app-wrapper">
            <main class="content-area">

div class="welcome-container">
   <div class="welcome-card">
       <div class="welcome-header">
           <h1 class="welcome-title">MoneyFlow</h1>
           <p class="welcome-subtitle">Secure Financial Transfers Made Simple</p>
       </div>

       <p class="welcome-description">
           MoneyFlow is a cutting-edge secure platform for swift and safe money transfers.
           Join our community of secure financial transactions.
       </p>

       <div class="button-group">
           <a href="/signin.php" class="btn btn-primary">Sign In</a>
           <a href="/signup.php" class="btn btn-secondary">Create Account</a>
       </div>
   </div>
/div>

       </main>
       <footer class="page-footer">
           <div class="footer-content">
               <p class="footer-text">&copy; 2026 MoneyFlow - Secure Fund Transfer Platform</p>
               <p class="footer-info">
                   Secure transactions powered by Team_9
               </p>
           </div>
       </footer>
   </div>
       <script src="/js/main.js?v=1773852893"></script>
/body>
/html>
nkit@ankit-HP-Z4-G4-Workstation:~$
```

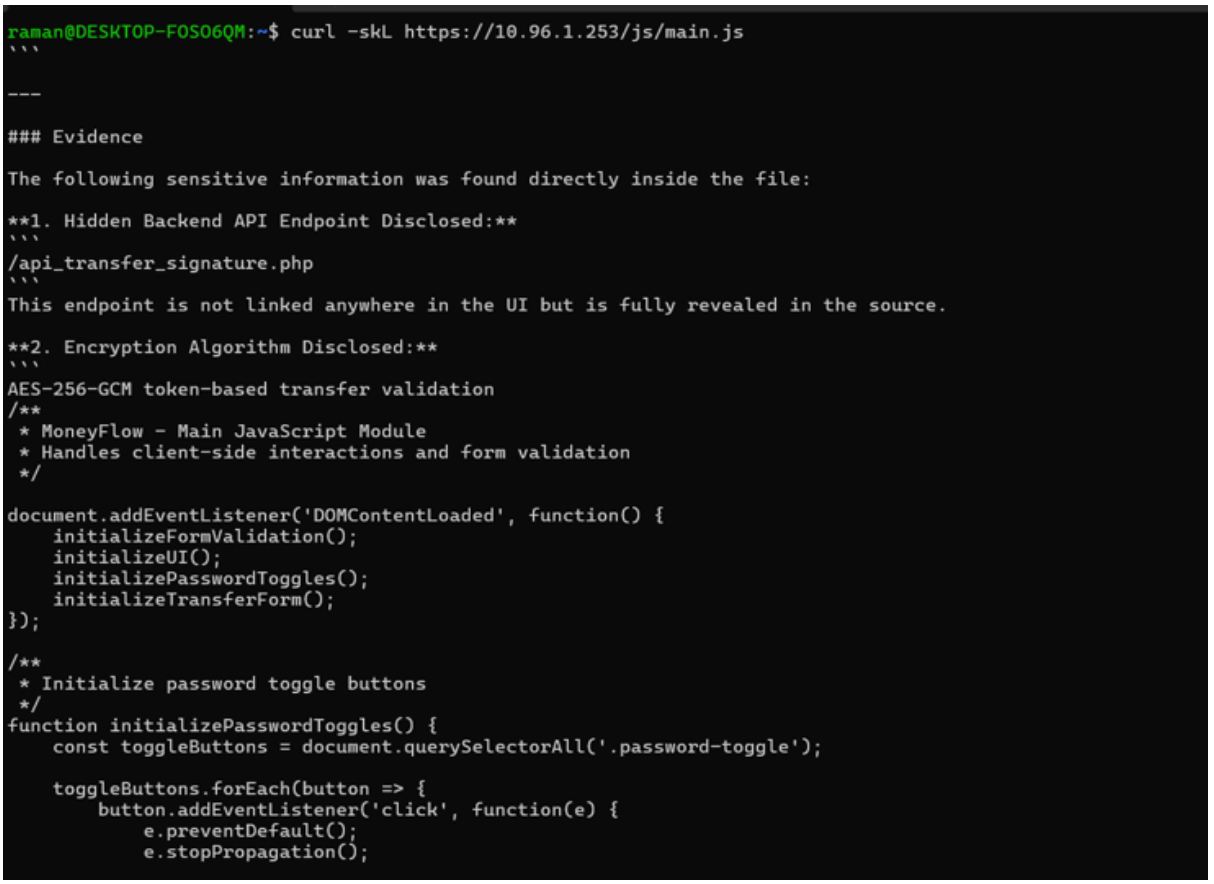
(12) Attack by – Team 03

Type of Attack – Sensitive source code exposed (Information disclosure — claiming that publicly accessible JavaScript file exposes sensitive security architecture details.)

Severity and Potential Impact – JavaScript files are client-side code — they are meant to be delivered to the browser. This is how the web fundamentally works. Knowing the endpoint name /api_transfer_signature.php is useless without a valid authenticated session and CSRF token. Knowing the encryption algorithm is AES-256-GCM reveals nothing exploitable since the encryption key never leaves the server. Developer comments describing the flow do not weaken the security — the security relies on server-side enforcement, not on the attacker's ignorance of how the system works.

Successful or Mitigated – This is not even a bug. Not applicable. No vulnerability was exploited. The attacker simply read a publicly served JavaScript file, which is normal browser behavior. Our security model does not depend on obscurity — every protection (session binding, CSRF validation, AES-256-GCM encryption, nonce anti-replay, rate limiting, DB intent binding) is enforced server-side regardless of what the client knows.

Validity – Invalid. The course professor also confirmed this is not a security issue. JavaScript is transferred to the browser by design — that is not information leakage. Security through obscurity is not a valid security mechanism, and our application was never designed to rely on it.



```
raman@DESKTOP-FOSO6QM:~$ curl -skL https://10.96.1.253/js/main.js
```

```
---

### Evidence

The following sensitive information was found directly inside the file:

**1. Hidden Backend API Endpoint Disclosed:**
```
/api_transfer_signature.php
```
This endpoint is not linked anywhere in the UI but is fully revealed in the source.

**2. Encryption Algorithm Disclosed:**
```
AES-256-GCM token-based transfer validation
/**
 * MoneyFlow - Main JavaScript Module
 * Handles client-side interactions and form validation
 */

document.addEventListener('DOMContentLoaded', function() {
    initializeFormValidation();
    initializeUI();
    initializePasswordToggles();
    initializeTransferForm();
});

/**
 * Initialize password toggle buttons
 */
function initializePasswordToggles() {
    const toggleButtons = document.querySelectorAll('.password-toggle');

    toggleButtons.forEach(button => {
        button.addEventListener('click', function(e) {
            e.preventDefault();
            e.stopPropagation();
```

**(13) Attack by –** Team 10

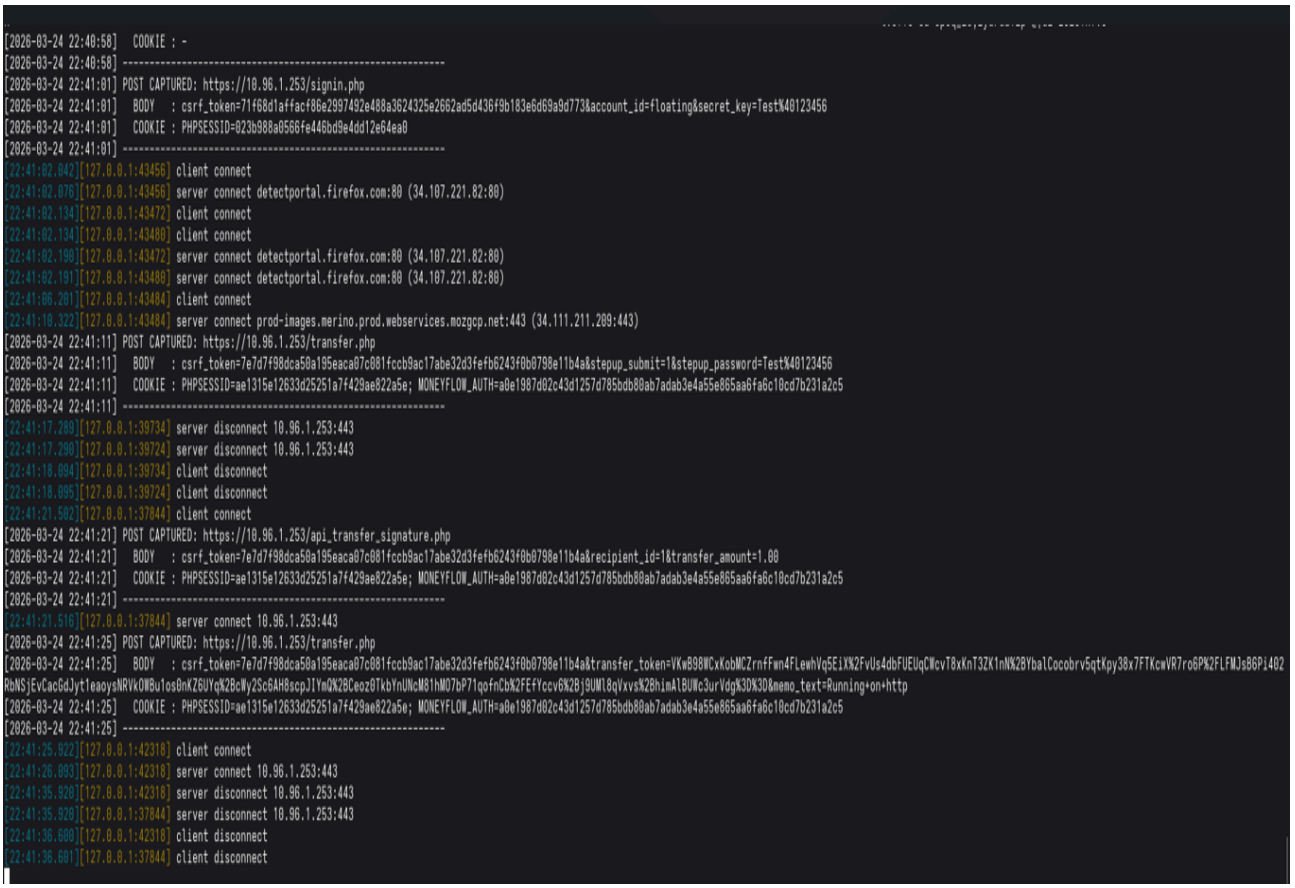
**Type of Attack –** SSL stripping — attempting to downgrade HTTPS connections to HTTP by intercepting and rewriting redirects.

**Severity and Potential Attack –** The attacker's own captured logs show all requests going to https:// URLs, not http://. No actual SSL stripping occurred. What the screenshot shows is a proxy tool (mitmproxy/Burp) intercepting HTTPS traffic after the attacker manually trusted their own proxy certificate — that is not an SSL strip, that is just running a proxy.

**Successful or Mitigated – This is not even a bug.** Not applicable. The attack was not successful. Our application sets HSTS headers (max-age=31536000; includeSubDomains; preload) in two places and enforces TLS 1.3 exclusively. The HTTP-to-HTTPS redirect is in place on port 8080. The attacker claims HSTS preload is missing, but HSTS preload only works with domain names — it is architecturally impossible for IP addresses by design. This is not specific to our application; every single team's application hosted on the college VMs has this same limitation since no one was provided domain names. Buying a domain and getting a CA-signed certificate was never part of the project requirements.

**Validity – Invalid.** The attacker conflated proxy interception with SSL stripping. The HSTS preload limitation on IP addresses is an infrastructure constraint common
```

to all teams, not a vulnerability in our application code. Our HSTS header still protects all returning visitors after the first HTTPS visit.



(B) Attack My team conducted on Other Applicants – Our Team conducted some attacks on other peers website, which are as follow –

(1) Attack 01 (against Team 01): Transaction failure - Money Transfer not functional -

- **Attack Strategy –** We attempted to perform a standard money transfer on Team 01's application. After logging in as a registered user with a balance of Rs. 100, we navigated to the transfer page and tried to send money to user CS25MTECH13009 (ID 33). The transfer was submitted with a valid amount and sufficient balance.

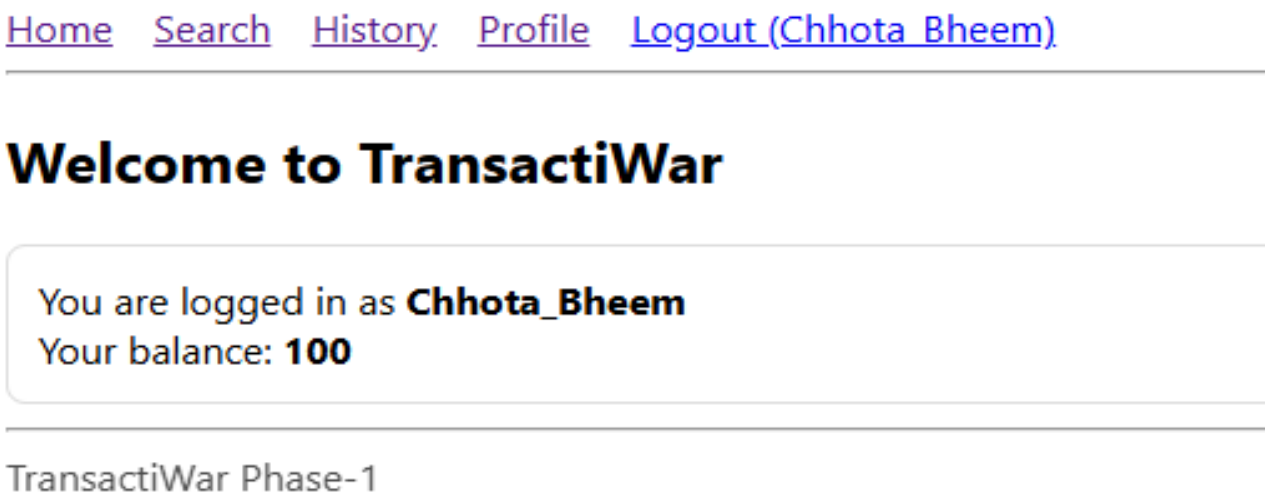
- **Vulnerability and Impact –** The application displayed "Transfer failed" despite the sender having sufficient funds. This is a core functionality bug — the primary feature of the application (money transfer) does not work. Users are unable to perform any transactions, which defeats the entire purpose of the platform. This indicates a broken transaction logic or a misconfigured backend that was never properly tested before deployment.

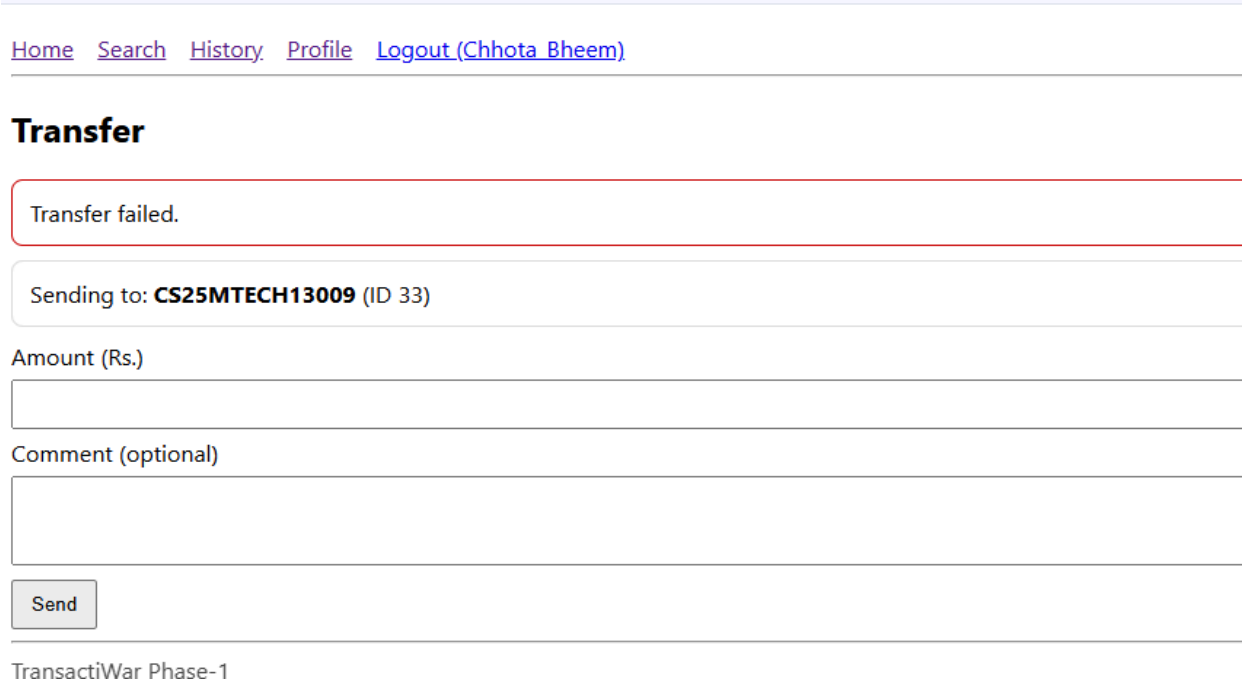
- **Security Observations (Strengths and Weakness) –**

  (a) Weakness – The transfer functionality, which is the most critical feature of the application, is completely non-functional. This suggests inadequate testing before deployment.

  (b) Weakness – The error message "Transfer failed" is generic and provides no useful feedback to the user about what went wrong, indicating poor error handling.

  (c) Strength – The application does have a login system and session management in place, as we were able to register, log in, and navigate to the transfer page without issues.





(2) Attack 02 (against Team 13):Global Rate limit on Registration - Self Inflicted DOS –

- **Attack Strategy –** We attempted to register a new account on Team 13's application. On navigating to the registration page, we filled in valid details (Operator ID, Email, Access Code) and submitted the form.

- **Vulnerability and Impact –** The application immediately displayed "CRITICAL ERROR: Too many registration attempts. Please wait an hour" — even though this was our first registration attempt. This indicates that the rate limiting on registration is implemented globally (shared counter across all users and IPs) rather than being scoped per IP address. The impact is significant — once any user or group of users triggers the rate limit threshold, no one else can register on the platform for an hour. This is essentially a self-inflicted Denial of Service on the registration endpoint. An attacker could deliberately trigger this limit with a few rapid requests and completely block all legitimate registrations for the cooldown period.

- **Security Observations (Strengths and Weakness) –**

  (a) Weakness – The rate limiting on registration is global instead of per-IP, which is a design flaw. Rate limits should track attempts per individual IP address so that one user's activity does not block others.
  (b) Weakness – The error message exposes the exact cooldown duration ("wait an hour"), which helps an attacker know precisely how long the lockout lasts**.**
  (c) Strength – The application does have rate limiting in place, which shows awareness of automated abuse. The implementation just needs to be scoped correctly to per-IP rather than global.
  (d) Strength – The registration page enforces strong password requirements (minimum 10 characters, uppercase + lowercase + digit + symbol).



Ambiguous Attack(which we are not sure that it is an attack) –

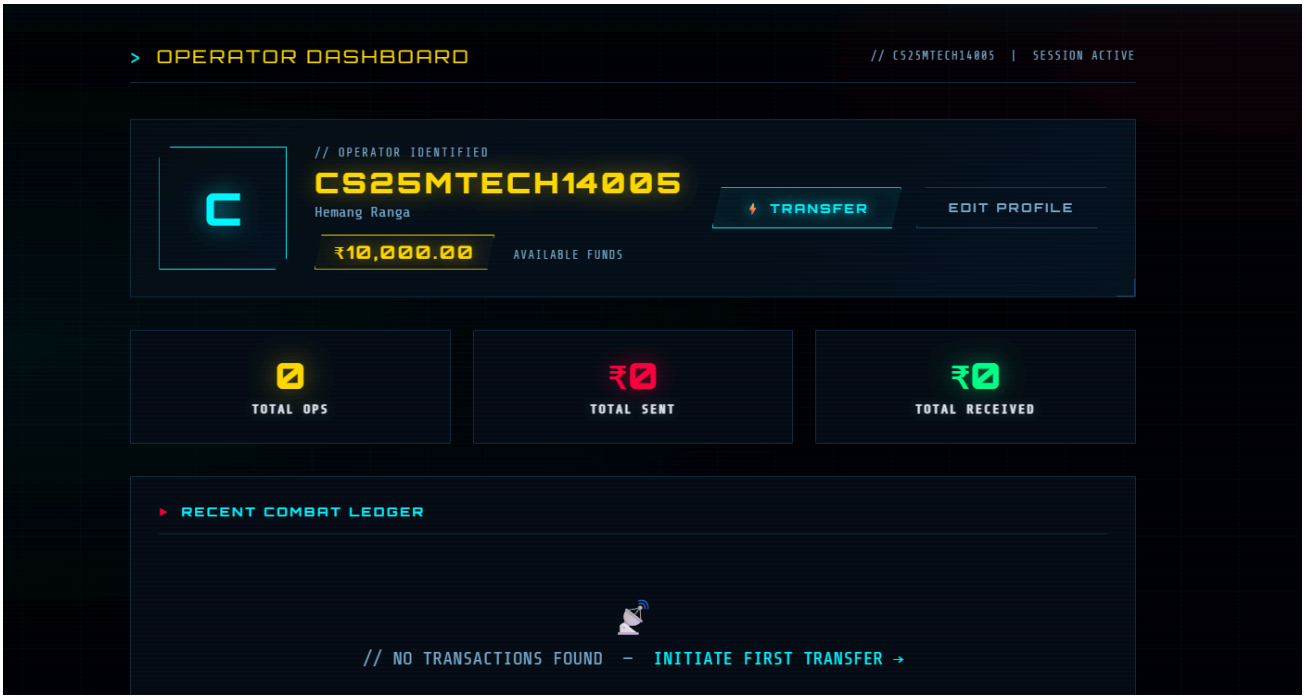
(3) Attack 03 (against Team 13): Wrong Balance –

- **Attack Strategy –** After successfully registering on Team 13's application (in a separate attempt when rate limiting was not blocking), we logged into the dashboard and observed the account balance.

- **Vulnerability & Impact –** The dashboard displayed an available balance of Rs. 10,000 for a newly registered account. As per the project specification, every new user should be credited with Rs. 100 only. This is a configuration or implementation error — the initial balance is set 100 times higher than the requirement. While not a direct security vulnerability, this is a significant deviation from the specified requirements. In a real-world scenario, incorrect initial balances could be exploited for unauthorized fund transfers — a user could immediately transfer Rs. 10,000 to another account, effectively generating money that should not exist.
- **Security Observations (Strengths & Weaknesses) –**

  (a) Weakness: The initial balance logic does not follow the project specification (Rs. 100), suggesting either a hardcoding error or a misconfigured database seed script. Lack of attention to such details raises questions about what other specification requirements may have been overlooked.

  (b) Strength: The application has a well-designed UI with a clean dashboard showing transaction stats (Total Ops, Total Sent, Total Received) and a recent transaction ledger, indicating effort was put into the frontend and user experience.



(3) Security Patching and Enhancements –

(A) Security Vulnerabilities identified in our Application –

While no team was able to successfully exploit any vulnerability in our application, we conducted our own internal review during the War Game phase and identified the following areas where our defenses could be further strengthened:

(a) **Display Errors Exposed During Deployment:** During initial deployment on the VM, PHP's display_errors was briefly enabled, which caused internal file paths to appear in error messages (e.g., /var/www/html/utilities/operations.php). While no attacker leveraged this, it could have aided reconnaissance.

(b) **File Upload Directory Permissions:** The avatar upload directory initially had incorrect ownership, causing move_uploaded_file() to fail with a "Permission denied" error. This was a deployment misconfiguration, not a code-level vulnerability, but it exposed internal paths through error messages.

(c) **HTTP to HTTPS Redirection Hardening:** While we had an HTTP-to-HTTPS redirect in place, we reviewed and strengthened the redirection logic to ensure no edge cases could allow plaintext communication.

(d) **HSTS on IP-Based Deployment:** We recognized that HSTS preload cannot protect IP-based deployments since preload lists only work with domain names. While this is an infrastructure limitation and not fixable without a domain name, we ensured our HSTS header with max-age=31536000; includeSubDomains; preload was correctly set to protect returning visitors.

(B) Measures Taken Post-War Game to Fix Vulnerabilities :

(a) **Disabled Display Errors:** Set display_errors = Off and log_errors = On in PHP configuration so that errors are logged server-side but never shown to users.

(b) **Fixed Upload Directory Permissions:** Corrected the ownership and permissions on the avatars upload directory to allow www-data to write files. Used a named Docker volume to preserve correct permissions across container restarts.

(c) **Strengthened HTTP Redirection:** Enhanced Apache configuration to ensure all HTTP traffic is properly redirected to HTTPS with no bypass possibilities.

(d) **Extended PHP Function Restrictions:** Expanded the disable_functions list to cover additional dangerous functions like curl_exec, curl_multi_exec, putenv, proc_nice, and proc_get_status.

(e) **Container Hardening:** Added resource limits (1 CPU, 512 MB RAM per container) to prevent resource exhaustion attacks, and ensured the database port is not exposed outside the Docker network.

(C) Before and After Comparison –

Security Aspect	Before Initial Deployment	After Post War Game
Error Display	display_errors briefly on during deployment, exposing internal paths.	display_errors = Off, errors logged server-side only
Upload Directory	Permission misconfiguration causing failures and error leakage	Upload Directory Permission misconfiguration causing failures and error leakage
HTTP Redirection	Basic redirect in place	Hardened redirection with no edge-case bypasses
PHP Disabled Function	Core dangerous functions disabled	Extended list covering additional execution and network functions
Container Resources	No explicit resource limit	1 CPU, 512 MB RAM cap per container preventing exhaustion
Database Exposure	Database accessible within Docker network	MySQL port explicitly not exposed to host
Security Headers	HSTS, CSP, X-Frame-Options set	All headers reviewed and strengthened, cache-control added for sensitive pages

(4) Lessons learned and Future Security Enhancements –

(A) Biggest Security Flaws Discovered and Lessons Learned –

(a) **Deployment Configuration Matters as Much as Code:** Our application code was secure, but deployment misconfigurations (file permissions, display_errors) briefly exposed internal information. The lesson is that security is not just about writing secure code — the deployment pipeline and server configuration need equal attention. A single misconfigured flag can undo layers of code-level protection.

(b) **Error Handling Must Be Silent in Production:** Verbose error messages that help during development become an information goldmine for attackers in production. We learned to always separate development and production configurations, with production environments logging errors internally and showing only generic messages to users.

(c) **Infrastructure Limitations Are Not Application Bugs:** The HSTS preload limitation on IP addresses taught us that some security features are only fully effective with

proper infrastructure (domain names, CA-signed certificates). Understanding the boundaries of each security mechanism helps in setting realistic expectations and planning compensating controls.

(d) **Most Reported "Attacks" Were Invalid:** A significant number of attack reports we received were fabricated, tested against non-existent endpoints, or based on fundamental misunderstandings of how our system works. This reinforced the importance of building security that does not rely on obscurity — our defenses held up because they were structurally sound, not because attackers did not know about them.

(B) How We Would Redesign the Application to Enhance Security Further –

(a) **Domain Name with CA-Signed Certificate:** Deploy with a proper domain name and a Let's Encrypt certificate to enable full HSTS preload protection, eliminating the first-visit HTTP vulnerability entirely.

(b) **Content Security Policy with Nonces:** Instead of relying solely on self in CSP, implement per-request nonces for any inline requirements, providing even more granular control over script execution.

(c) **Web Application Firewall (WAF):** Add a WAF layer (such as ModSecurity with OWASP Core Rule Set) in front of the application to catch common attack patterns before they reach application code.

(d) **Automated Security Testing in CI/CD:** Integrate tools like static analysis and dependency scanning into the build pipeline so that vulnerabilities are caught before deployment rather than after.

(e) **Two-Factor Authentication:** Add TOTP-based two-factor authentication for an additional layer of protection beyond passwords, especially for sensitive operations like money transfers.

(f) **Centralized Logging with Alerting:** Instead of database-only logging, push audit logs to a centralized logging system with real-time alerting on suspicious patterns like repeated failed logins or unusual transfer activity.

(C) Best Practices and Real World Implications –

(a) **Defense in Depth Works:** Our experience validated that layering multiple independent security controls is the most effective strategy. When one layer is probed, the others remain standing. No single mechanism should be the sole line of defense.

(b) **Validate on the Server, Never Trust the Client:** Every input validation and security check must be enforced server-side. Client-side checks are for user experience only

— they can always be bypassed. Our DB intent binding pattern for transfers is a practical example of this principle.

(c) **Principle of Least Privilege:** Restricting database user permissions to only SELECT, INSERT, UPDATE, and DELETE, disabling dangerous PHP functions, and running containers as non-root users all follow the principle of giving each component only the minimum access it needs. In a real-world scenario, this limits the blast radius of any breach.

(d) **Secure Defaults Over Secure Options:** Security mechanisms should be on by default, not something that needs to be enabled. Our application enforces HTTPS, requires strong passwords, and applies rate limiting without any user or administrator action — this is how production systems should behave.

(e) **Regular Security Audits Are Essential:** Even a well-built application benefits from periodic security reviews. The War Game phase itself served as a valuable peer audit, and in real-world development, regular penetration testing and code reviews are essential to maintaining a strong security posture over time.

Contributions

Throughout the project, all five team members contributed equally and collaborated closely across all phases — from initial development and deployment to the War Game attack/defense activities and report preparation. While each member took primary ownership of a specific domain, everyone participated in code reviews, testing, and security discussions as a team.

Manan Kannasey (CS25MTECH11028) — Authentication & Security:
Implemented the user authentication system including Argon2id password hashing, login/registration flows with rate limiting, CAPTCHA integration, and the custom session management system with token binding and expiration logic. During the War Game phase, actively participated in attacking other teams' applications and identifying vulnerabilities.

Hemang Ranga (CS25MTECH14005) — DevOps & Core Operations:
Set up the Docker environment (Dockerfile, docker-compose, Apache configuration), handled server hardening (TLS 1.3, security headers, PHP restrictions, unprivileged container execution), and managed deployment on the college VM. During the War Game phase, actively participated in probing and attacking other teams' applications for security weaknesses.

Rohit B (CS25MTECH14012) — Database & Logging:
Designed the complete database schema including all tables, CHECK constraints, foreign keys, indexes, and MySQL scheduled cleanup events for automatic data retention. Implemented the activity logging and audit trail system, built the CSRF token generation and validation mechanism, and developed the rate limiting infrastructure across all endpoints. During the War Game phase, actively participated in conducting attacks on other teams' applications and analyzing their security implementations.

Yash Vardhan (CS25MTECH14016) — Frontend & Security Patching:
Built the complete frontend — all HTML pages, CSS styling, form layouts, and user interface components. Ensured all forms include CSRF tokens and proper output encoding for XSS prevention. During the War Game phase, took the lead in monitoring incoming attack reports and promptly patching any design flaws or configuration issues that arose to keep our application secure.

Deependra Singh (CS25MTECH14017) — Transactions & JavaScript:
Developed the two-step money transfer flow with AES-256-GCM token encryption, the client-side JavaScript logic, input validation module, file upload security pipeline, and the transfer signature API endpoint. During the War Game phase, actively participated in attacking other teams' applications and documenting the vulnerabilities found.

During the War Game phase, all members equally participated in analyzing incoming attack reports, defending our application, conducting attacks on other teams' applications, and preparing this security assessment report.

References –

(1) PHP Official Documentation — PHP 8.2 language reference, PDO, session handling, GD library, and built-in functions.
https://www.php.net/docs.php

(2) PHP password_hash and Argon2id — Documentation on password hashing functions and Argon2id algorithm support in PHP.
https://www.php.net/manual/en/function.password-hash.php

(3) PHP random_bytes — Cryptographically secure random byte generation for session tokens and CSRF tokens.
https://www.php.net/manual/en/function.random-bytes.php

(4) PHP openssl_encrypt (AES-256-GCM) — Authenticated encryption for transfer token generation.
https://www.php.net/manual/en/function.openssl-encrypt.php

(5) PHP PDO Prepared Statements — Parameterized queries for SQL injection prevention.
https://www.php.net/manual/en/pdo.prepared-statements.php

(6) PHP GD Library — Image processing and re-encoding for secure file upload handling.
https://www.php.net/manual/en/book.image.php

(7) MySQL 8.0 Reference Manual — Database schema design, InnoDB transactions, row-level locking, scheduled events, and CHECK constraints.
https://dev.mysql.com/doc/refman/8.0/en/

(8) Apache HTTP Server Documentation — Virtual host configuration, mod_rewrite, mod_headers, mod_ssl, and security hardening.
https://httpd.apache.org/docs/2.4/

(9) Docker Documentation — Dockerfile best practices, docker-compose configuration, multi-container orchestration, and health checks.
https://docs.docker.com/

(100 OWASP Top 10 (2021) — Reference for identifying and mitigating common web application security vulnerabilities including SQL injection, XSS, CSRF, and broken authentication.
https://owasp.org/www-project-top-ten/

(11) OWASP Cheat Sheet Series — Practical guidance on session management, password storage, input validation, file upload security, and HTTP security headers.
https://cheatsheetseries.owasp.org/

(12) Mozilla Developer Network (MDN) Web Docs — HTML5, CSS3, JavaScript, Fetch API, and HTTP security headers reference.

https://developer.mozilla.org/

(13) Mozilla Observatory / Security Headers — Reference for Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, and other HTTP security headers.
https://observatory.mozilla.org/

(14) OpenSSL Documentation — Self-signed TLS certificate generation for HTTPS configuration.
https://www.openssl.org/docs/

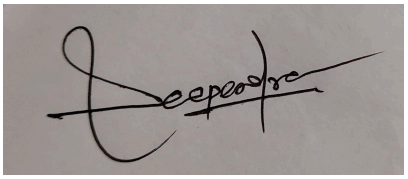
(15) NIST Special Publication 800-63B — Digital Identity Guidelines for password storage and authentication best practices, including memory-hard hashing recommendations.
https://pages.nist.gov/800-63-3/sp800-63b.html

Anti-Plagiarism Statement –

We certify that this assignment/report is our own work, based on our personal study and/or research, and that we have acknowledged all material and sources used in its preparation, whether books, articles, packages, datasets, reports, lecture notes, or any other document, electronic or personal communication. We also certify that this assignment/report has not previously been submitted for assessment/project in any other course lab, except where specific permission has been granted from all course instructors involved, or at any other time in this course, and that we have not copied in part or whole or otherwise plagiarized the work of other students and/or persons. We pledge to uphold the principles of honesty and responsibility at CSE@IITH. In addition, we understand our responsibility to report honor violations by other students if we become aware of them.

Deependra Singh<CS25MTECH14017>
Yash Vardhan<CS25MTECH14016>
Rohit B<CS25MTECH14012>
Hemang Ranga<CS25MTECH14005>
Manan Kannasey<CS25MTECH11028>

Date – 26-03-2026



Yash Vardhan
Rohit B
Hemang Ranga
Manan Kannasey